β

This project is currently under beta-test, until it is validated by a fair number of students or a decent delay has elapsed.

- *Please report any typo, incoherence, inconsistency, error, using the form https://tally.so/r/3lVDJo.*
- *Once you're done with this project, please fill in the Beta Test Feedback form to share your overall experience and make sure you get properly rewarded: https://tally.so/r/Bzv7x4.*
- *A dedicated Slack channel exists to exchange on beta-test projects: #_world_beta_test.*
- *A FAQ page dedicated to beta-test projects is available here.*
- *The project currently awards 0 XP, on purpose; XP will be granted once the beta-test is over and conclusive.*





Piscine Deep Learning - 2

Vision

Summary: Today, you will teach a network to see: convolutions, pooling, and a full convolutional network for image classification.

Version: 1.0

Contents

I	Foreword	2
II	General Rules	3
III	Exercise 00	5
IV	Exercise 01	6
V	Exercise 02	7
VI	Exercise 03	8
VII	Exercise 04	9
VIII	Submission and peer-evaluation	10

Chapter I

Foreword

A fully connected network treats an image as a flat list of pixels and throws away everything we know about it: that nearby pixels are related, that a pattern is the same wherever it appears.

The **convolution** fixes that. Today you will load a real image dataset, understand what a convolution and a pooling layer do to a picture, assemble a **convolutional neural network** (CNN), train it to classify images, evaluate it properly, and finally reuse a network someone else already trained (**transfer learning**).

Chapter II

General Rules

- You must submit your modules from a computer in the cluster or using a virtual machine:
  - You can choose the operating system for your virtual machine.
  - Your virtual machine must have all the necessary software to complete your project. This software must be installed and properly configured.
- Alternatively, you can use the cluster computers directly if the necessary tools are available.
  - Ensure that you have enough space in your session to install all required dependencies for the modules (use `goinfre` if your campus provides it).
  - Everything must be installed before the evaluations. The datasets are downloaded through `torchvision`; download them once before your defense.
- We encourage you to create test programs for your project, even though these tests **do not need to be submitted and will not be graded**. They will help you easily test your work and your peers' work. These tests will be particularly useful during your defense. During the defense, you are free to use your own tests and/or those of the peer you are evaluating.
- Submit your work to your assigned Git repository. Only the work in the Git repository will be graded. If Deepthought is assigned to grade your work, it will occur after peer evaluations. If an error occurs in any section of your work during Deepthought's grading, the evaluation will be terminated.
- You must use Python version 3.10 or higher.
- The only external libraries allowed for this module are `PyTorch` (`torch`), `torchvision` and `matplotlib` (only where a plot is explicitly requested). No `numpy`, no `scikit-learn`.
- Your library imports must be explicit. For example, you must use `import torch`. Importing libraries using `from torch import *` is not allowed and will result in a score of `0` for the exercise.
- Global variables are not allowed.

- All your functions must have documentation (___doc___) and your code must follow the norm (`pip install flake8`, `alias norminette=flake8`).
- Each program must have its `main` and not be a simple script, and you should seed the random generator with `torch.manual_seed(42)` for reproducibility.
- By Odin, by Thor! Use your brain!!!

Chapter III

Exercise 00

	Exercise00
Exercise 00: Load images	
Directory: *ex*00/	
Files to Submit: `load_images.py`	
Authorized: `torch, torchvision, matplotlib`	

Write a program that loads the **FashionMNIST** dataset using `torchvision.datasets`, applies a `ToTensor` transform, and prints:

- the number of training and test samples,
- the shape of a single image,
- the number of classes.

As a (non-graded) bonus you may display a grid of a few images with their labels using `matplotlib`.

Expected output:

```
$>python load_images.py | cat -e
Training samples: 60000$
Test samples: 10000$
Image shape: torch.Size([1, 28, 28])$
Number of classes: 10$
$>
```



The first run downloads the dataset. Make sure it is already downloaded before an evaluation.

Chapter IV

Exercise 01

	Exercise01
Exercise 01: Convolution and pooling	
Directory: *ex*01/	
Files to Submit: `convolution.py`	
Authorized: `torch, torchvision, matplotlib`	

Before stacking them, understand what a convolution does to the size of an image. Take one image from FashionMNIST (shape `[1, 28, 28]`), add a batch dimension, then apply:

- a `nn.Conv2d` with 4 output channels and a 3×3 kernel (no padding),
- a `nn.MaxPool2d` with a 2×2 window.

Print the shape at each step.

Expected output:

```
$>python convolution.py | cat -e
Input shape : torch.Size([1, 1, 28, 28])$
After conv : torch.Size([1, 4, 26, 26])$
After pool : torch.Size([1, 4, 13, 13])$
$>
```



A 3×3 convolution with no padding removes one pixel on each side ($28 \to 26$). The pooling then halves each spatial dimension ($26 \to 13$). Being able to predict these shapes by hand is essential.

Chapter V

Exercise 02

	Exercise02
Exercise 02: Build a CNN	
Directory: *ex*02/	
Files to Submit: `cnn.py`	
Authorized: `torch, torchvision`	

Assemble a real convolutional network: at least two `Conv2d` → `ReLU` → `MaxPool2d` blocks followed by a fully connected head that outputs one score per class. Train it on FashionMNIST and print the accuracy on the test set.

A correct CNN should comfortably pass `0.85` test accuracy.

Expected output (values will vary):

```
$>python cnn.py | cat -e
Epoch 1 | train loss: 0.5123$
Epoch 2 | train loss: 0.3401$
Epoch 3 | train loss: 0.2876$
Test accuracy: 0.89$
$>
```



`If training on the full dataset is too slow on your machine, you may train on a subset, but your test accuracy must still be clearly above chance.`

Chapter VI

Exercise 03

	Exercise03
Exercise 03: Evaluate	
Directory: *ex*03/	
Files to Submit: `evaluate.py`	
Authorized: `torch, torchvision`	

Accuracy alone hides where a model fails. Load your trained model (or retrain it) and, on the test set, compute **with torch only**:

- the 10×10 **confusion matrix**,
- the per-class accuracy.

Print the shape of the confusion matrix and the per-class accuracy.

Expected output (values will vary):

```
$>python evaluate.py | cat -e
Confusion matrix shape: torch.Size([10, 10])$
Per-class accuracy:$
  class 0: 0.86$
  class 1: 0.97$
  ...$
  class 9: 0.95$
Overall accuracy: 0.89$
$>
```



You are not allowed to use scikit-learn here. Build the confusion matrix yourself: it is just a count of (true label, predicted label) pairs.

Chapter VII

Exercise 04

	Exercise04
Exercise 04: Transfer learning	
Directory: *ex*04/	
Files to Submit: `transfer.py`	
Authorized: `torch, torchvision`	

Training a big network from scratch is expensive. Instead, load a model **pretrained** on ImageNet from `torchvision.models` (for example `resnet18`), freeze its feature extractor, replace its final classification layer to match your number of classes, and train only that new head on a small dataset (for example a subset of CIFAR-10).

Print the accuracy obtained. Because most of the network is already trained, you should reach a good accuracy after very few epochs.

Expected output (values will vary):

```
$>python transfer.py | cat -e
Loaded pretrained resnet18$
Frozen feature extractor, training new head (10 classes)$
Epoch 1 | train loss: 0.9124$
Epoch 2 | train loss: 0.6321$
Validation accuracy: 0.78$
$>
```



`Freezing means setting requires_grad = False on the pretrained parameters so the optimizer only updates your new head.`

Chapter VIII

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.



`The evaluation process will happen on the computer of the evaluated group.`